

Nonparametric Workshop

Riobamba, Ecuador

Jeffrey S. Racine

McMaster University

Wednesday, April 19, 2023

Slide Pro-Tips

- Link to slides - jeffreyracine.github.io/RBworkshop (case sensitive) ([Google Translate](#))
- View **full screen** by pressing the F key (press the Esc key to revert)
- Access **navigation menu** by pressing the M key (navigation menu X to close)
- **Advance** using arrow keys
- **Zoom** in by holding down the Alt key in Windows, Opt key in macOS or Ctrl key in Linux, and clicking on any screen element (Alt/Opt/Ctrl click again to zoom out)
- Use **copy to clipboard** button for R code blocks (upper right in block) to copy and paste into R/RStudio
- **Export to a PDF** by pressing the E key (wait a few seconds, then print [or print using system dialog], enable landscape layout, then save as PDF - press the E key to revert)
- Enable drawing tools - chalk **board** by pressing the B key (B to revert), notes **canvas** by pressing the C key (C to revert), press the Del key to erase, press the D key to **download drawings**

Welcome!

- The website with install info for R, RStudio, and $\text{T}_{\text{E}}\text{X}$ is
<https://jeffreyracine.github.io/RBworkshop>
- The GitHub repository with example code is
<https://github.com/JeffreyRacine/RBworkshop>
- No seas tímido, siéntete libre de hacer preguntas
- ¡Vámonos!

Welcome!

- You will be guided through basic nonparametric kernel methods using R
- You will also, if patient, be introduced to recently released tools for conducting reproducible research
- No background knowledge of either nonparametric analysis or the R programming language is required
- All software is available for *free* and is *open source*
- For more detailed descriptions see Racine ([2019](#)), Racine ([2012](#)), and Hayfield and Racine ([2008](#))

Overview

- I hope that you leave this workshop armed with a modern set of data-analytic tools
- Let's briefly discuss the following:
 - [What is R?](#)
 - [What is RStudio?](#)
 - [What is Kernel Regression?](#)
 - [What is Quarto?](#)

Background Data Types

R and Data Types

! Important

Pre-cast data prior to analysis so R functions can do their job

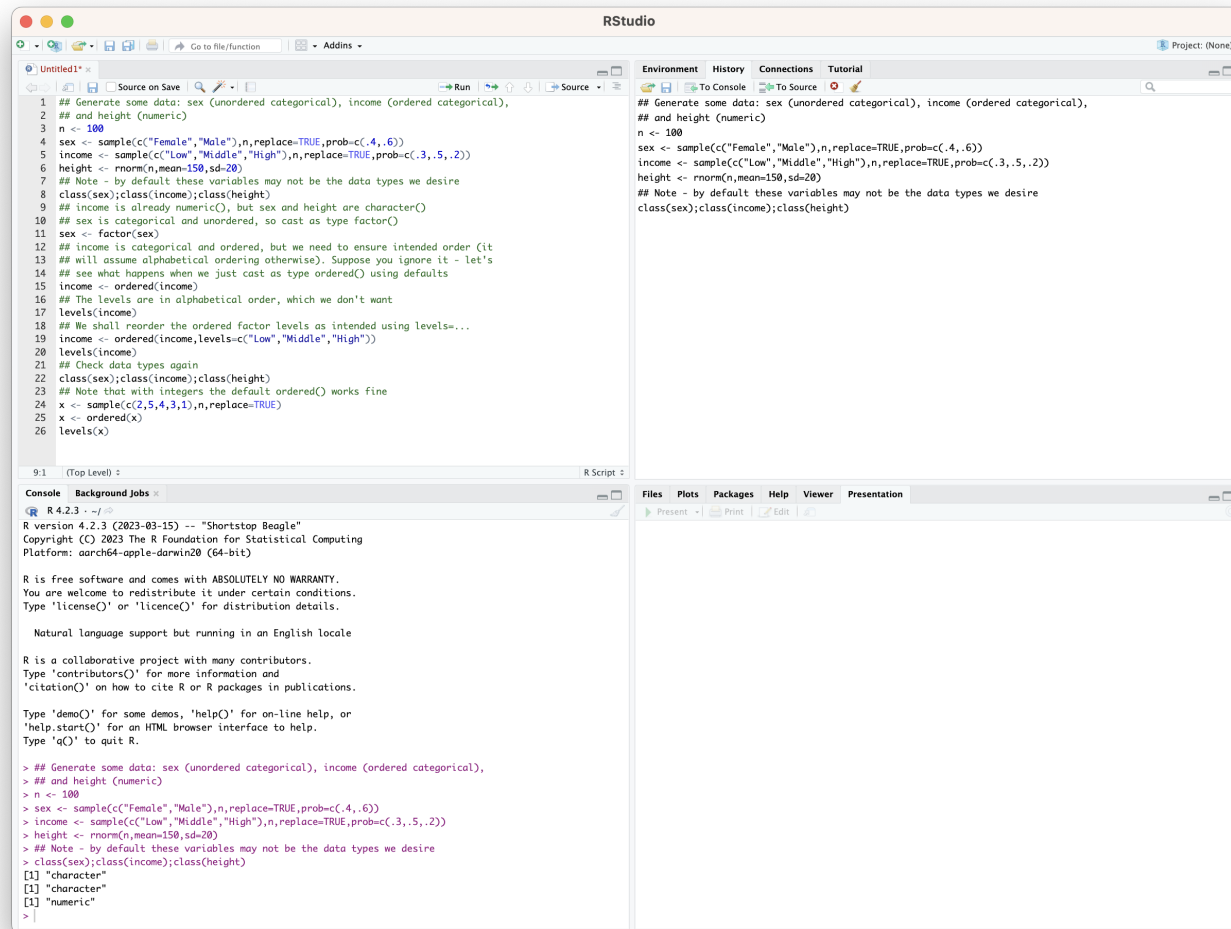
- It is *crucial* to understand different data types in R
- There are three R functions we will use: `numeric()`, `factor()`, and `ordered()`
- These correspond to data that are *numbers*, *unordered categories*, and *ordered categories*, respectively
- Pay careful attention to the following example

R and Data Types

- Below is an example of how we *cast* our data in R *prior* to analyzing it
- Copy and paste this into a new R file in RStudio, then run it line-by-line

```
1  ## Generate some data: sex (unordered categorical), income (ordered categorical),
2  ## and height (numeric)
3  n <- 100
4  sex <- sample(c("Female", "Male"), n, replace=TRUE, prob=c(.4, .6))
5  income <- sample(c("Low", "Middle", "High"), n, replace=TRUE, prob=c(.3, .5, .2))
6  height <- rnorm(n, mean=150, sd=20)
7  ## Note - by default these variables may not be the data types we desire
8  class(sex); class(income); class(height)
9  ## income is already numeric(), but sex and height are character()
10 ## sex is categorical and unordered, so cast as type factor()
11 sex <- factor(sex)
12 ## income is categorical and ordered, but we need to ensure intended order (it
13 ## will assume alphabetical ordering otherwise). Suppose you ignore it - let's
14 ## see what happens when we just cast as type ordered() using defaults
15 income <- ordered(income)
16 ## The levels are in alphabetical order, which we don't want
17 levels(income)
18 ## We shall reorder the ordered factor levels as intended using levels=...
19 income <- ordered(income, levels=c("Low", "Middle", "High"))
20 levels(income)
21 ## Check data types again
22 class(sex); class(income); class(height)
23 ## Note that with integers the default ordered() works fine
24 x <- sample(c(2, 5, 4, 3, 1), n, replace=TRUE)
25 x <- ordered(x)
26 levels(x)
```


RStudio R Editor Interface

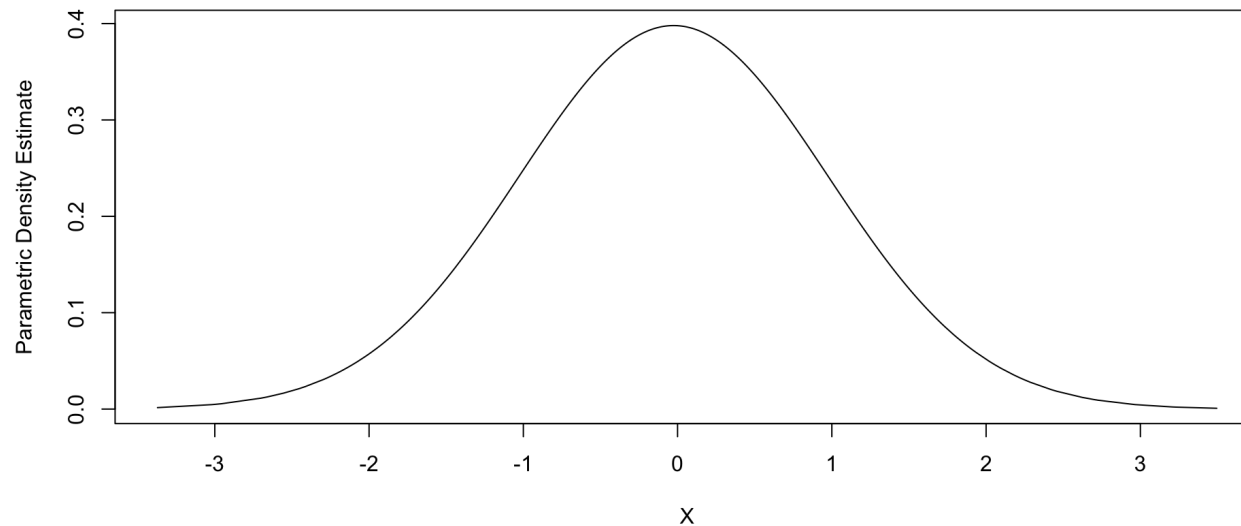


Background Density Estimation

Parametric or Nonparametric?

Suppose we need to estimate the density $f(x)$ of some numeric random variable X

```
1  ## Let's simulate a random numeric sample from the normal distribution
2  set.seed(42)
3  n <- 1000
4  x <- rnorm(n)
5  ## Let's sort the data so we can graph x versus dnorm(x,...) using lines (type="l")
6  x <- sort(x)
7  ## Since we simulated the data, let's plot the true, known, parametric density
8  ## (we can't do this with actual data because the density of such data is, in
9  ## general, unknown)
10 plot(x, dnorm(x, mean=mean(x), sd=sd(x)), type="l", ylab="Parametric Density Estimate", xlab="X")
```



Parametric or Nonparametric?

- The R function `shapiro.test(x)` tests for normality

Shapiro-Wilk normality

test: `x`

Test statistic	P value
0.9988	0.767

- This is *simulated normal* data, and we fail to reject the null

Warning

What if we got the null density wrong (i.e., what if we rejected the null)?

Parametric or Nonparametric?

- Now let's test normality for actual data, `eruptions`

Shapiro-Wilk normality test:
`eruptions`

Test statistic	P value
0.8459	9.036e-16 * *
	*

Warning

Oh oh... I guess we rule out the normal parametric model... what next?

Parametric or Nonparametric?

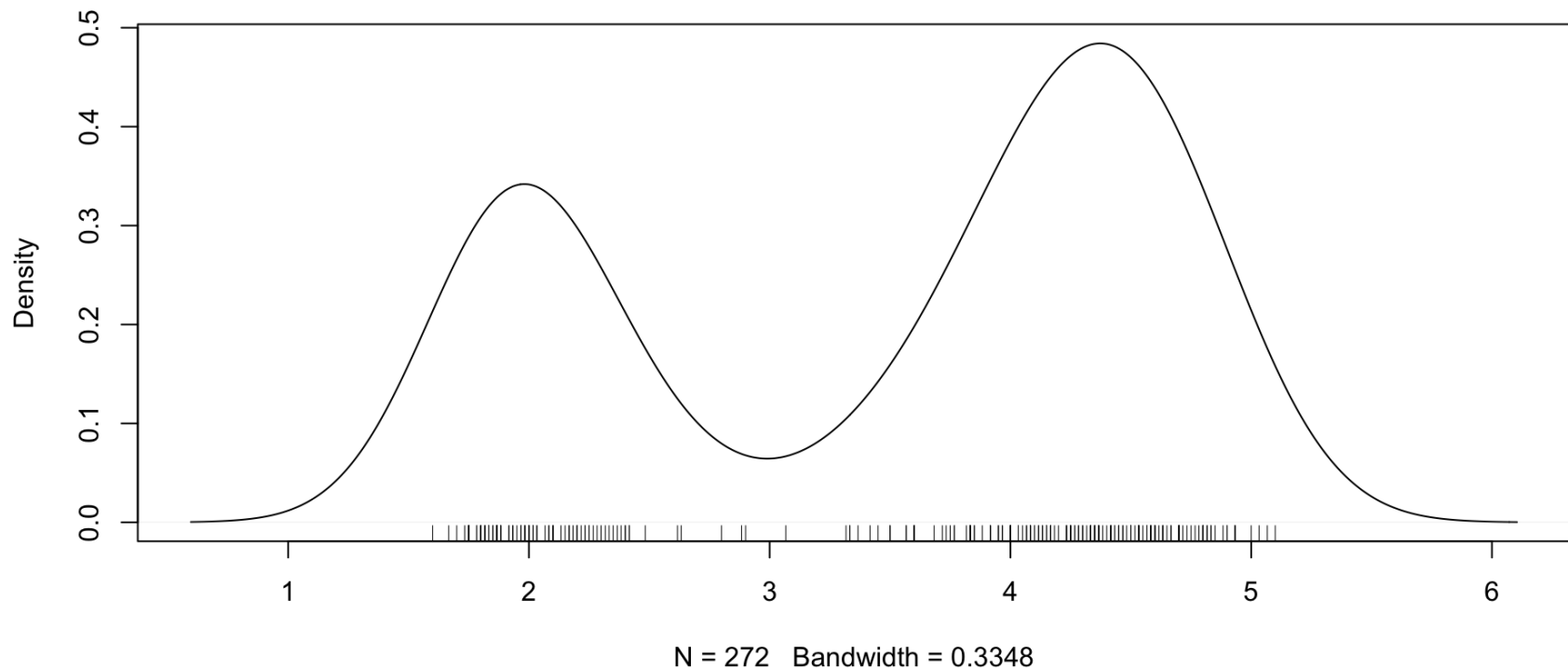
- Actual data ($X_i, i = 1, \dots, n$) is rarely drawn from the simple parametric distributions used in *sampling theory*
- Sampling theory describes *summary statistics*, e.g., *averages* of actual data, or *averages* of squared deviations, such as

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i \quad \text{or} \quad \hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \hat{\mu})^2$$

- But we need to estimate the *density of the actual data* (i.e., $f(x)$), not the *density of some summary statistic* (i.e., $f(\hat{\mu})$)

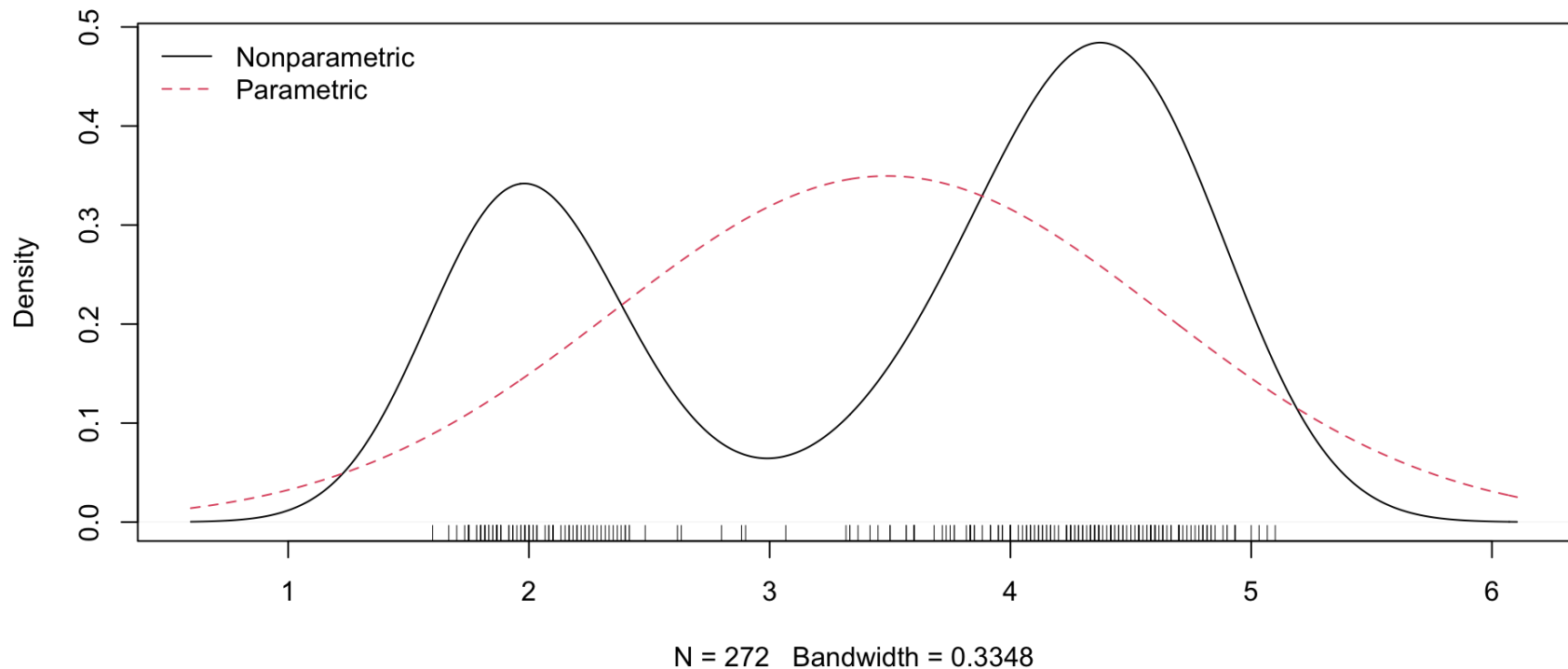
Parametric or Nonparametric?

- Below is a nonparametric density estimate and data *rug* for the **eruptions** data (it is bi-modal *and* asymmetric)



Parametric or Nonparametric?

- Below is a nonparametric and *normal* parametric estimate (recall the normal parametric model has been *rejected*)



Parametric or Nonparametric?

- Here lies the crux of the parametric problem
- We write down some parametric model that is drawn from a *dense* space of functions (their number is *uncountable*)
- One very important assumption is *this parametric model is the true model*, and all model properties (unbiasedness, consistency etc.) depend *crucially* on this being the case
- But if you are serious, you immediately *test* your model for correct specification
- What if the parametric model is rejected, as it often is?

Nonparametric Essentials

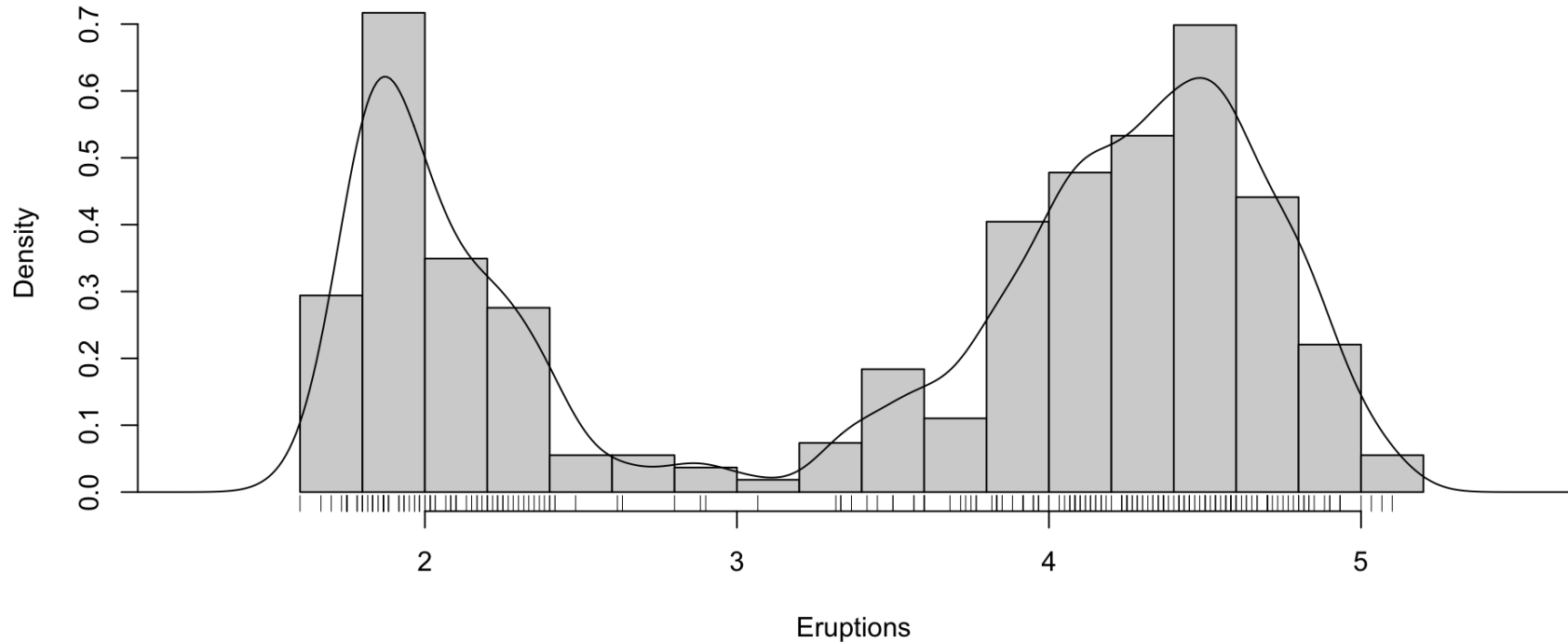
Numeric Data

Numeric Data

- Parametric methods require the user to make *very strong assumptions* about the *data generating process* (DGP)
- We just considered density estimation, but the exact same issue plagues all parametric analysis (i.e., regression, etc.)
- We will consider, instead, nonparametric *kernel* estimators
- A *kernel* is simply a weight function, which for *numeric* data we denote by $K(Z_i)$ where $Z_i = (x - X_i)/h$
- We give *higher* weight to observations close to x (i.e., small Z_i) than to those lying further away (i.e., large Z_i)

Non-Smooth or Smooth?

- Below are non-smooth and smooth nonparametric density estimates for the *numeric* variable **eruptions**



Non-Smooth or Smooth?

- The non-smooth nonparametric histogram density estimator is

$$f_H(x) = \frac{1}{nh} \sum_{i=1}^n \mathbf{1}(X_i \text{ is in the same bin as } x)$$

- This estimator has drawbacks, including
 - it is not particularly *efficient* in the statistical sense
 - the estimator's discontinuity presents obstacles if derivatives are required ($df_H(x)/dx$ is 0 or undefined)

Smooth Univariate Density Estimation

- Consider a symmetric smooth kernel function that satisfies $K(z) \geq 0$ and $\int_{-\infty}^{\infty} K(z) dz = 1$
- The smooth nonparametric kernel density estimator is

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - X_i}{h}\right)$$

- This estimator is dominant in applied settings, though *boundary bias* can be an issue (simple corrections exist, see `npuniden.boundary()` in the `np` package)

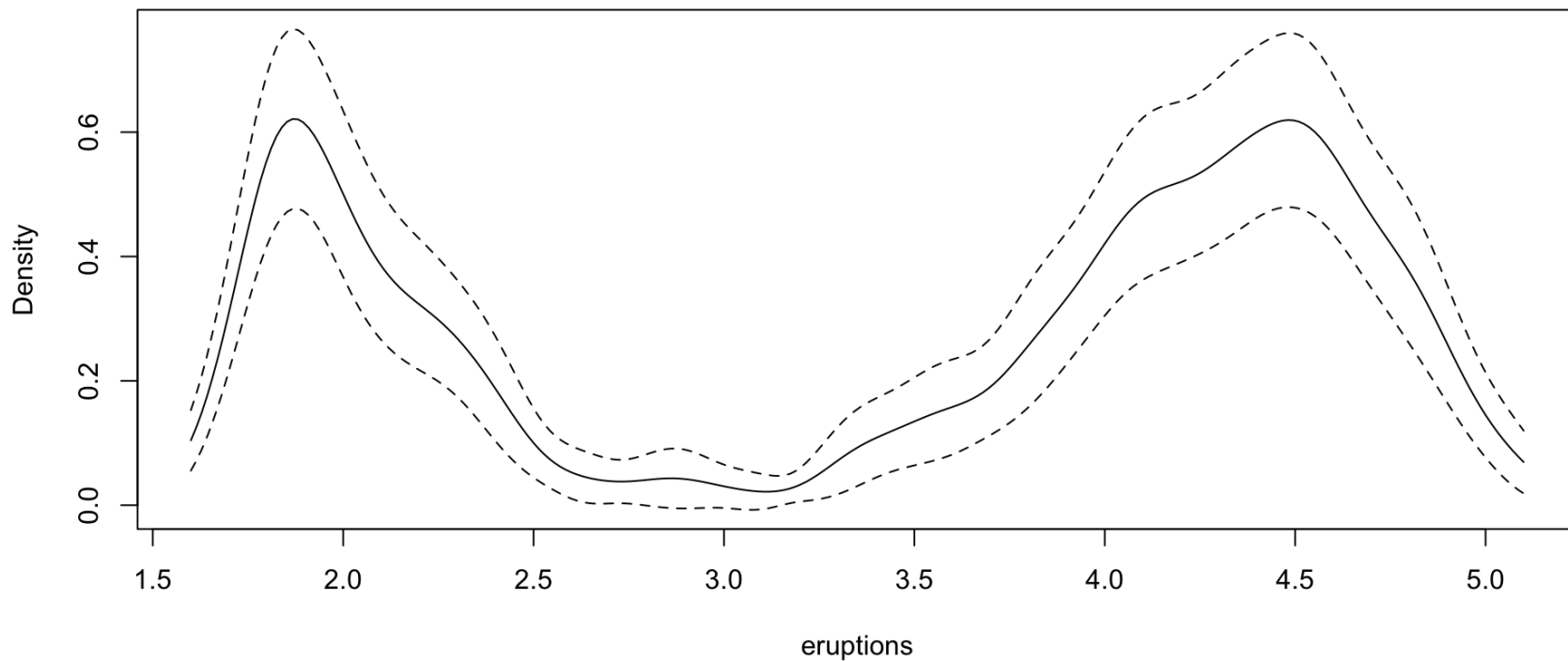
Smooth Univariate Density Estimation

! Important

- The kernel function $K(z)$ is relatively unimportant (it imparts *smoothness* on the estimate, where smooth functions are *continuously differentiable* functions)
 - The bandwidth h is crucial (it governs the bias-variance trade-off)
-
- *Closeness* is determined by a *bandwidth*, denoted h
 - We choose h to minimize *square error risk* and trade off *bias* for *variance* for the sample at hand
 - To accomplish this we use *data-driven* methods for selecting h (e.g., least-squares cross-validation)

Smooth Univariate Density Estimation

```
1 library(np)
2 data(faithful)
3 fhat <- npudens(~eruptions,data=faithful)
4 plot(fhat,neval=250,plot.errors.method="bootstrap")
```



Smooth Joint Density Estimation

- So far we have considered univariate densities ($X \in \mathbb{R}^1$)
- Let $X \in \mathbb{R}^q$ denote a numeric vector of dimension q
- Let $f(x) = f(x_1, x_2, \dots, x_q)$ denote a joint PDF evaluated at $x = (x_1, x_2, \dots, x_q)'$
- Let $\{X_1, X_2, \dots, X_n\}$ represent n draws of a numeric random vector X , where the i th draw is denoted by $X_i = (X_{i1}, X_{i2}, \dots, X_{iq})$

Smooth Joint Density Estimation

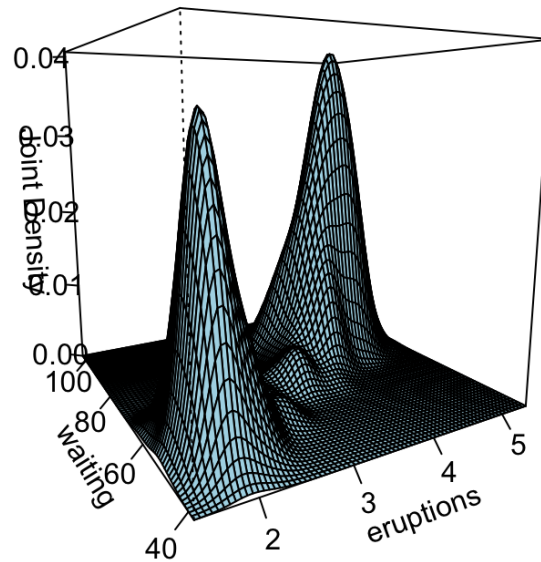
- The multivariate kernel density estimator is

$$\hat{f}(x) = \frac{1}{nh_1 \dots h_q} \sum_{i=1}^n K \left(\frac{x_1 - X_{i1}}{h_1}, \dots, \frac{x_q - X_{iq}}{h_q} \right)$$

- $K(\cdot)$ is a *multivariate* kernel (typically the *product* of univariate kernels - we have lots of flexibility here)
- We continue to use data-driven methods (typically cross-validation) for bandwidth selection when $q \geq 1$ (optimal bandwidths differ across variables, i.e., $h_1 \neq h_2$, etc.)

Smooth Joint Density Estimation

```
1 library(np)
2 data(faithful)
3 fhat <- npudens(~eruptions+waiting,data=faithful)
4 plot(fhat,theta=330,xtrim=-0.05,neval=75,view="fixed",main="")
```



Nonparametric Essentials

Categorical Data

Categorical Variables

- We considered the density of continuously distributed random variables $X \in \mathbb{R}$, but we also deal with the probability function of categorical variables $X \in \mathcal{D}$ where $\mathcal{D}$ is a discrete set
- We might presume a parametric model for the probability function of $X \in \mathcal{D}$ but we face exactly the same issue we faced before
- The non-smooth categorical counterpart to the histogram is called the *frequency* or *empirical* probability estimator and, like the histogram, has some drawbacks

Univariate Probability Estimation

- Let $X \in \mathcal{D} = \{0, 1, \dots, c - 1\}$ be categorical
- X can be *unordered* (`factor()` in R) or *ordered* (`ordered()` in R)
- The nonparametric *frequency* estimator of $p(x)$ is

$$\begin{aligned} p_n(x) &= \frac{\#X_i \text{ equal to } x}{n} \\ &= \frac{1}{n} \sum_{i=1}^n \mathbf{1}(X_i = x) \end{aligned}$$

Univariate Probability Estimation

- The *unordered* nonparametric kernel estimator of $p(x)$ is

$$\hat{p}(x) = \frac{1}{n} \sum_{i=1}^n L(X_i, x, \lambda)$$

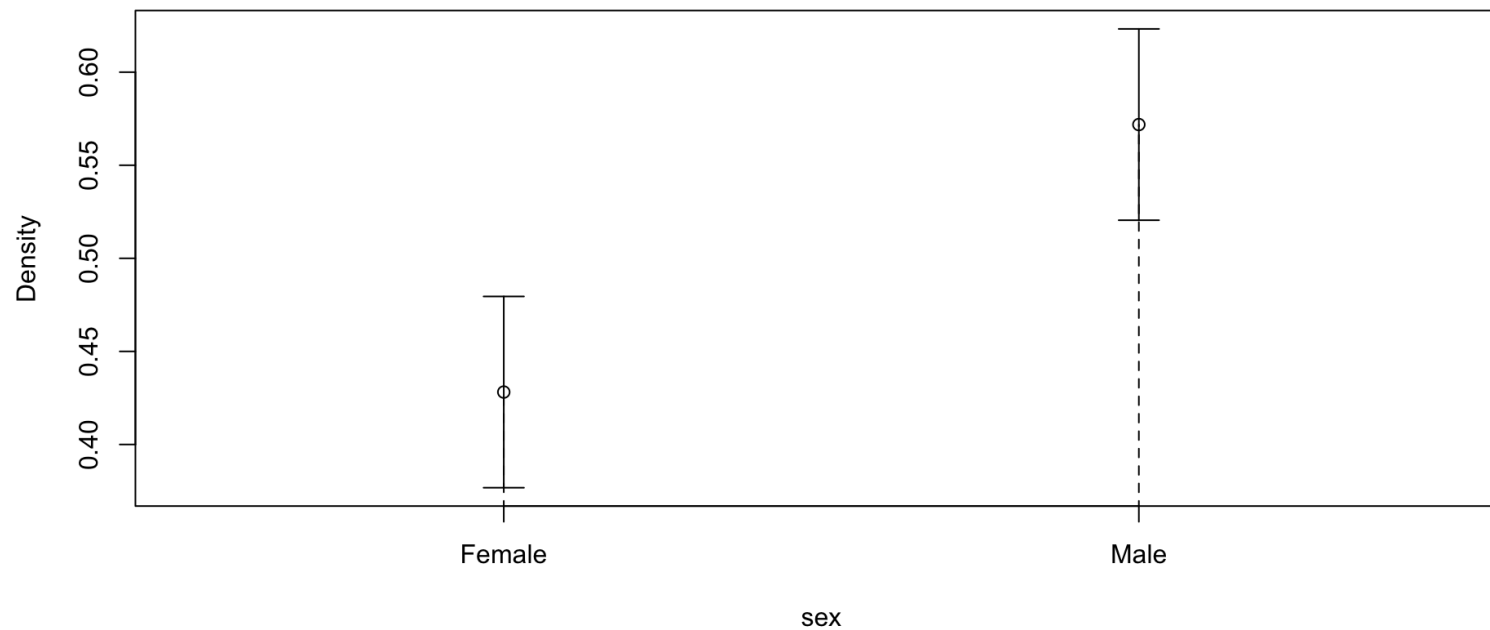
- $L(\cdot)$ is an *unordered* kernel function given by

$$L(X_i, x, \lambda) = \begin{cases} 1 - \lambda & \text{if } X_i = x \\ \lambda/(c - 1) & \text{otherwise} \end{cases}$$

- λ is a *smoothing parameter* (counterpart to *bandwidth* h)

Example - Unordered Probability

```
1 n <- 250
2 set.seed(42)
3 sex <- sample(c("Female", "Male"), n, replace=TRUE, prob=c(.4, .6))
4 sex <- factor(sex)
5 phat <- npudens(~sex)
6 plot(phat, plot.errors.method="bootstrap")
```



Univariate Probability Estimation

- Let $X \in \mathcal{D} = \{0, 1, \dots, c - 1\}$, $c \geq 2$, be *ordered*
- The *ordered* nonparametric kernel estimator of $p(x)$ is

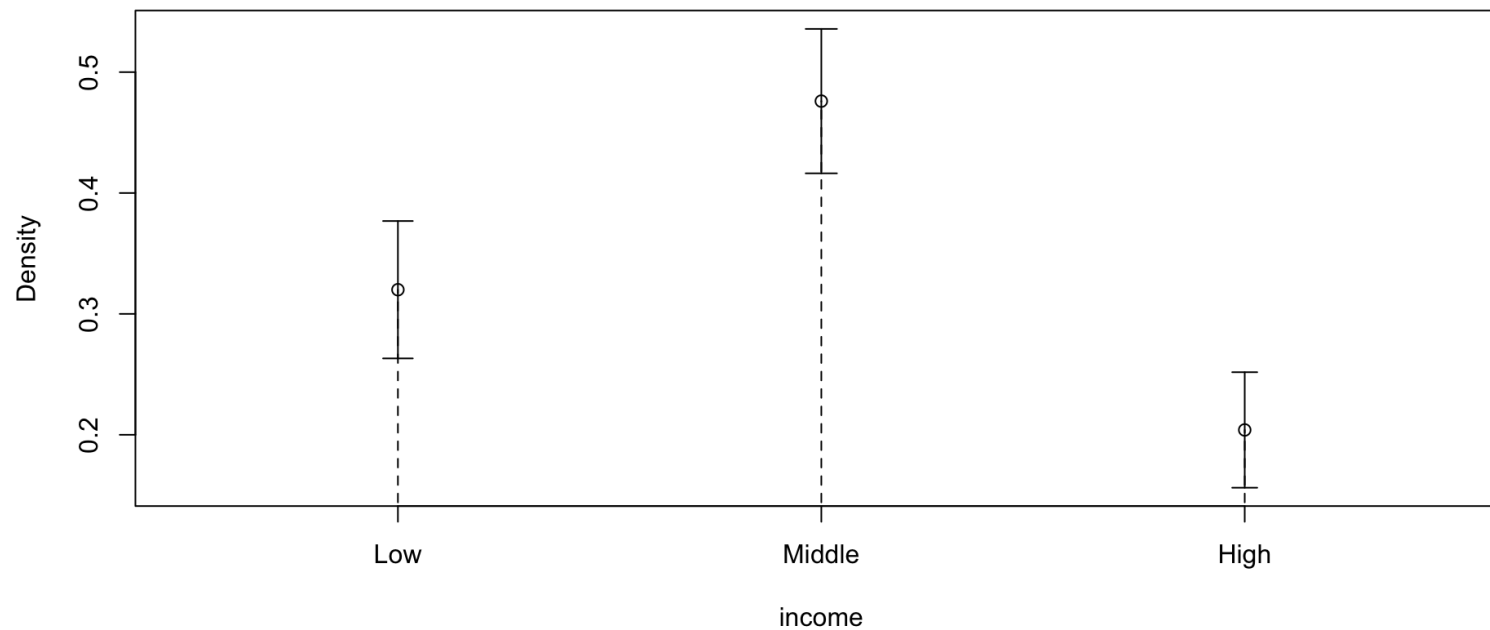
$$\hat{p}(x) = \frac{1}{n} \sum_{i=1}^n l(X_i, x, \lambda)$$

- $l(X_i, x, \lambda)$ is an *ordered* kernel function given by

$$l(X_i, x, \lambda) = \frac{\lambda^{d_{xi}}}{\Lambda_i}$$

Example - Ordered Probability

```
1 n <- 250
2 set.seed(42)
3 income <- sample(c("Low", "Middle", "High"), n, replace=TRUE, prob=c(.3, .5, .2))
4 income <- ordered(income, levels=c("Low", "Middle", "High"))
5 phat <- npudens(~income)
6 plot(phat, plot.errors.method="bootstrap")
```



Nonparametric Essentials

Mixed Data

Mixed Data Density Estimation

- Statisticians know that $f(x)$ and $p(x)$ are both called *density* functions (the difference lies in their *measure* - the latter uses *counting* measure), so we adopt $f(x)$
- Suppose you have a joint density defined over mixed data types, say, one numeric ($X^c \in \mathbb{R}$) and one unordered ($X^d \in \mathcal{D}$ with cardinality c)
- We would like to model their joint density function $f(x) = f(x^c, x^d)$, where the superscripts c and d denote continuous and discrete data types, respectively, and where $x = (x^c, x^d) \in \mathbb{R} \times \mathcal{D}$

Mixed Data Density Estimation

- The kernel estimator of $f(x^c, x^d)$ is

$$\hat{f}(x^c, x^d) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{X_i^c - x^c}{h}\right) L(X_i^d, x^d, \lambda)$$

- $K(\cdot)$ and $L(\cdot)$ are (univariate) *numeric* and *unordered* kernel functions, respectively
- If you had one numeric and one *ordered* variable you would use the ordered kernel $l(X_i^d, x^d, \lambda)$ above
- Now let's consider the *general* multivariate mixed data density case

Mixed Data Density Estimation

- In general multivariate settings, the probability density function $f(x)$ might use some combination of q numeric, r unordered, and s ordered variable types (x and X_i are $q + r + s$ -vectors)
- The (product) kernel for estimating the joint density function is

$$\prod_{j=1}^q h_j^{-1} K\left(\frac{x_j^c - X_{ij}^c}{h_j}\right) \prod_{j=1}^r L(X_{ij}^u, x_j^u, \lambda_j^u) \prod_{j=1}^s l(X_{ij}^o, x_j^o, \lambda_j^o)$$

- All we are doing here is using the *appropriate kernel function* for each data type

Mixed Data Density Estimation

- Let $\gamma = (h_1, \dots, h_q, \lambda_1^u, \dots, \lambda_r^u, \lambda_1^o, \dots, \lambda_s^o)$
- Call the product kernel function on the previous slide the *generalized* kernel, i.e., let the expression be written as

$$K_\gamma(X_i, x) = \prod_{j=1}^q [\dots] \prod_{j=1}^r [\dots] \prod_{j=1}^s [\dots]$$

- With x and X_i being $q + r + s$ vectors, we write $\hat{f}(x)$ as

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n K_\gamma(X_i, x)$$

Mixed Data Density Estimation

- The key point is that *once you cast your data*, then the R functions in the **np** package know *exactly* what to do
- They *automatically* use the appropriate kernel for the appropriate data type (numeric, factor, ordered)
- The data-driven bandwidth methods adjust automatically to the data type
- Methods for inference (confidence intervals, significance testing in regression) do the same
- Let's consider a quick example

Example - Mixed Data Density

- We use Wooldridge's [wage1](#) data and consider two variables, one numeric ([lwage](#)) and one ordered ([numdep](#))
- [lwage](#) is the logarithm of an individual's average hourly earnings, and [numdep](#) is their number of dependents
- The number of observations in each cell is tabulated in [Table 1](#)

Table 1: Counts of number of dependants
present in 526 households by cell

numdep	0	1	2	3	4	5	6
counts	252	105	99	45	16	7	2

- We estimate $\hat{f}(lwage, numdep)$ and plot it in [Figure 1](#)

Example - Mixed Data Density

- Below is the R code to estimate and plot the joint density (we use the `plot3D` package and reformat the data to render this plot)

```
1 library(np)
2 library(plot3D)
3 data(wage1)
4 bw <- npudensbw(~lwage+ordered(numdep),data=wage1)
5 numdep.seq <- with(wage1,sort(unique(numdep)))
6 lwage.seq <- with(wage1,seq(min(lwage),max(lwage),length=50))
7 wage1.eval <- expand.grid(numdep=ordered(numdep.seq),lwage=lwage.seq)
8 fhat <- fitted(npudens(bws=bw,newdata=wage1.eval))
9 ## Hack since scatter3D converts ordered 0-6 to numeric 1-7
10 scatter3D(as.numeric(wage1.eval[,1])-1,wage1.eval[,2],fhat,
11           ylab="Log-Wage",
12           xlab="Number of Dependants",
13           zlab="Joint Density",
14           ticktype="detailed",
15           angle=15,
16           box=TRUE,
17           type="h",
18           grid=TRUE,
19           col="blue",
20           colkey=FALSE)
```

Example - Mixed Data Density

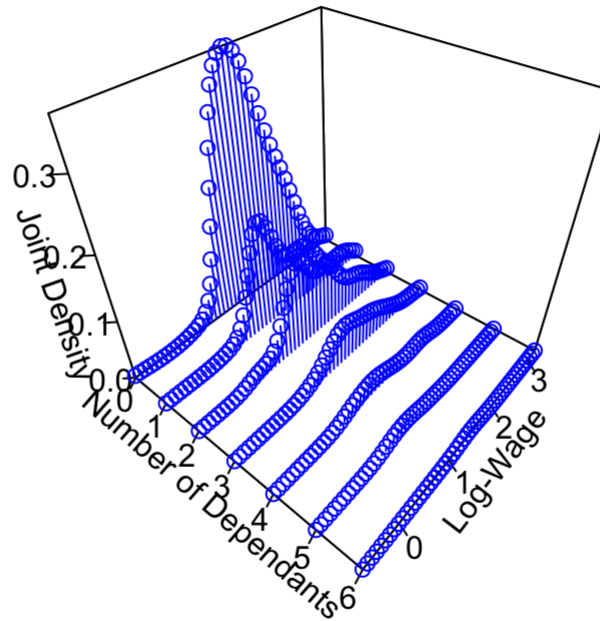


Figure 1: Mixed-data bivariate kernel density estimate for the joint PDF of lwage (numeric) and numdeps (ordered)

Nonparametric Regression

Nonparametric Regression

- The conditional mean function $g(x) := \mathbb{E}(Y|X = x)$ for numeric $Y \in \mathbb{R}$ is *defined* as

$$g(x) = \int y \frac{f(y, x)}{f(x)} dy = \frac{m(x)}{f(x)} \quad (1)$$

- Let's start with $X \in \mathbb{R}^1$ (i.e., one numeric predictor)
- We aim to estimate the unknown regression model $y = g(x) + \varepsilon$ without assuming that, e.g., $g(x) = \beta_0 + \beta_1 x$

Nonparametric Regression

- Regression is *hard* because $f(y, x)$ and $f(x)$ are unknown
- They can be *consistently* estimated using $\hat{f}(y, x)$ and $\hat{f}(x)$
- For continuously distributed $Y \in \mathbb{R}$, the *local constant* kernel regression estimator replaces the *unknown* $f(y, x)$ and $f(x)$ with *known* $\hat{f}(y, x)$ and $\hat{f}(x)$ in (1), hence

$$\hat{g}(x) = \int y \frac{\hat{f}(y, x)}{\hat{f}(x)} dy = \frac{\hat{m}(x)}{\hat{f}(x)} \quad (2)$$

Nonparametric Regression

- For $X \in \mathbb{R}^1$, we estimate $g(x)$ by replacing the unknown $f(x)$ in (1) with

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K \left(\frac{x - X_i}{h} \right),$$

- and replacing the unknown $f(y, x)$ in (1) with

$$\hat{f}(y, x) = \frac{1}{nh_y h} \sum_{i=1}^n K \left(\frac{y - Y_i}{h_y} \right) K \left(\frac{x - X_i}{h} \right)$$

Nonparametric Regression

- Some mathematical simplification of $\hat{g}(x)$ leads to

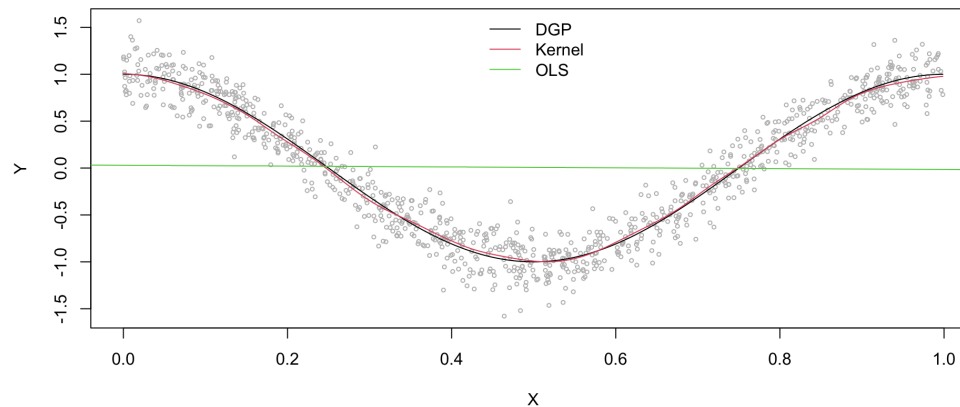
$$\hat{g}(x) = \sum_{i=1}^n Y_i \left\{ \frac{K\left(\frac{X_i - x}{h}\right)}{\sum_{i=1}^n K\left(\frac{X_i - x}{h}\right)} \right\} = \sum_{i=1}^n Y_i W_i(x)$$

- With mixed data multivariate X (i.e., x and X_i are $q + r + s$ -vectors) we replace $K(\cdot)$ with $K_\gamma(X_i, x)$
- The estimated nonparametric regression function is

$$y = \hat{g}(x) + \hat{\epsilon} \quad (3)$$

Simulated Data Illustration

```
1 library(np)
2 set.seed(42)
3 n <- 1000
4 x <- sort(runif(n))
5 dgp <- cos(2*pi*x)
6 y <- dgp + rnorm(n,sd=0.25*sd(dgp))
7 ghat <- npreg(y~x)
8 plot(x,y,cex=0.5,col="grey",xlab="X",ylab="Y")
9 lines(x,dgp)
10 lines(x,fitted(ghat),col=2)
11 abline(ghat.ols <- lm(y~x),col=3)
12 legend("top",c("DGP", "Kernel", "OLS"),col=1:3,lty=1,bty="n")
```



Simulated Data Illustration: OLS

How does the simple parametric linear model `ghat.ols` do for this simulated data?

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.03148	0.04529	0.695	0.4872
x	-0.04666	0.07966	-0.5858	0.5582

Fitting linear model: $y \sim x$

Observations	Residual Std. Error	R^2	Adjusted R^2
1000	0.7338	0.0003437	-0.000658



Warning

Oh oh... this linear parametric model truly sucks... the nonparametric model has an R^2 of 0.9411 (the parametric model has an R^2 of 0.0003437 and a negative adjusted R^2)

Marginal Effects

- The univariate X *marginal effects function* is simply the first partial derivative function, and is defined as

$$\begin{aligned}\beta(x) &= \frac{dg(x)}{dx} = \frac{f(x)m'(x) - m(x)f'(x)}{f^2(x)} \\ &= \frac{m'(x)}{f(x)} - g(x)\frac{f'(x)}{f(x)}\end{aligned}\tag{4}$$

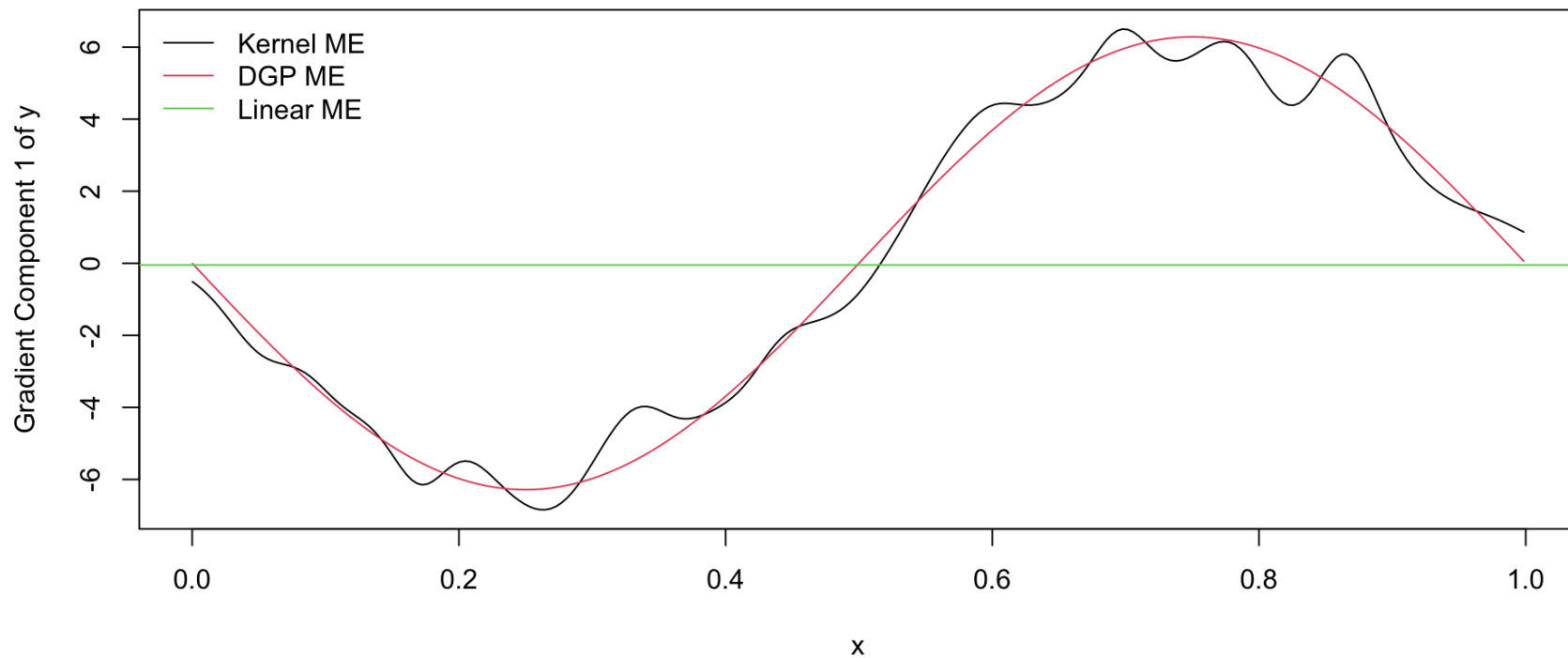
- We construct $\hat{\beta}(x)$ by replacing $f(x)$, $m'(x)$, $m(x)$, and $f'(x)$ in (4) with $\hat{f}(x)$, $\hat{m}'(x)$, $\hat{m}(x)$, and $\hat{f}'(x)$

Marginal Effects

- Recall the definitions of the conditional mean and marginal effects functions, $g(x)$ and $\beta(x)$
- They involve unknown joint and marginal densities and their derivatives, which are functions of x and y
- If you were tasked with estimating $\beta(x)$, it is hard to justify the assumption that $\beta(x)$ is *constant* (i.e., some constant β that is *not* a function of x like $\beta(x)$)
- But this is *exactly* what is assumed for the popular linear regression model (i.e., $\beta(x) = dy/dx = \beta_1$, a *constant*)

Example - Simulated Marginal Effects

```
1 plot(ghat,gradients=TRUE,neval=250)
2 lines(x,-2*pi*sin(2*pi*x),col=2)
3 abline(h=coef(ghat.ols)[2],col=3)
4 legend("topleft",c("Kernel ME","DGP ME","Linear ME"),col=1:3,lty=1,bty="n")
```



Mixed Data Marginal Effects

- Consider a multivariate conditional mean function $g(x)$ where x is composed of q continuous, r unordered, and s ordered predictors
- As in the previous univariate example, if the j th predictor is continuous, i.e., $x_j \in \mathbb{R}$, then $\hat{\beta}_j(x)$ is the first partial derivative function of $\hat{g}(x)$ with respect to the j th predictor, i.e.,

$$\hat{\beta}_j(x) = \frac{\partial \hat{g}(x)}{\partial x_j} = \frac{\hat{m}^{(j)}(x)}{\hat{f}(x)} - \hat{g}(x) \frac{\hat{f}^{(j)}(x)}{\hat{f}(x)}$$

Multivariate Mixed-Data Marginal Effects

- If the j th predictor is unordered in $\mathcal{D} = \{a, b, c\}$, then $\hat{\beta}_j(x)$ is the difference between $\hat{g}(x)$ when $x_j^u = b$ versus $\hat{g}(x)$ when $x_j^u = a$, and between $\hat{g}(x)$ evaluated at $x_j^u = c$ versus at $x_j^u = a$ (a is the *base* category)

$$\hat{\beta}_j(x) = \hat{g}(x_{(-j)}, x_j = l) - \hat{g}(x_{(-j)}, x_j = a), l = b, c$$

- If the j th predictor is ordered we have two options, namely, to take differences as in the unordered case, or to take differences between *successive* elements of the ordered set (i.e., between a and b then between b and c)

Multivariate Regression

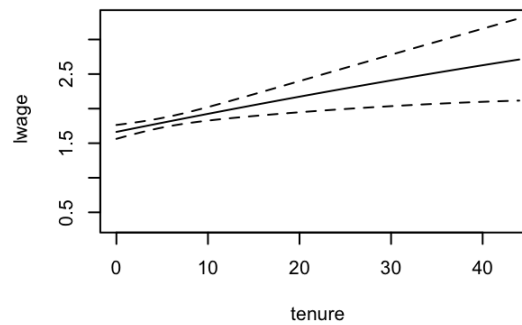
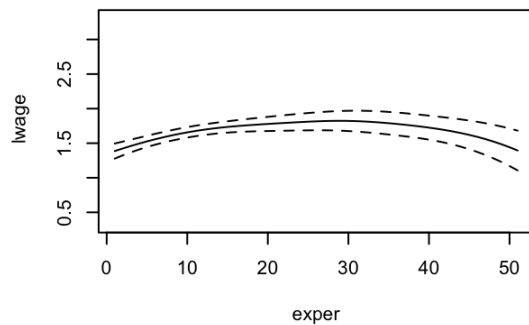
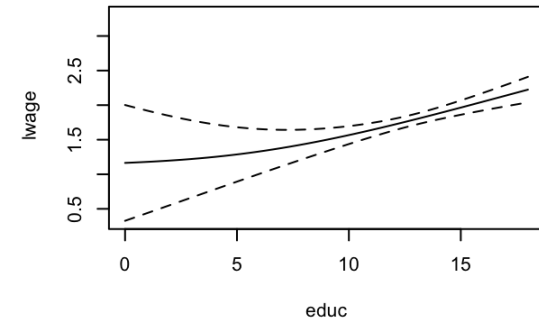
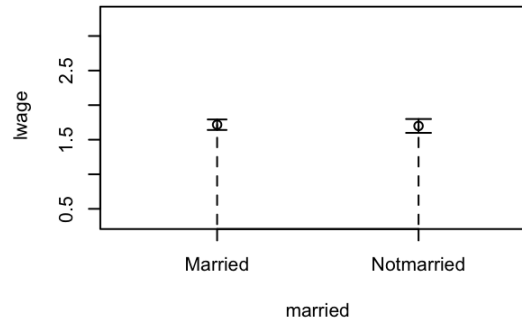
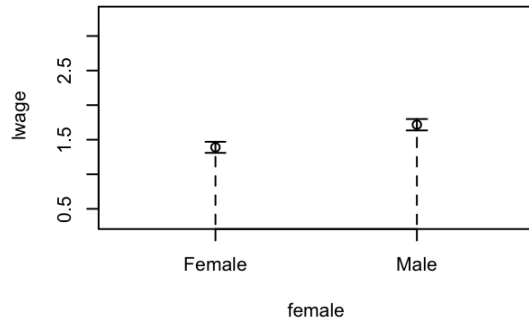
- We use Wooldridge's `wage1` data containing numeric and categorical predictors ([link to wage1 description](#))
- We regress `lwage` on categorical predictors `female` and `married` and numeric predictors `educ`, `exper` and `tenure`
- The formula for the regression models is
`lwage~female+married+educ+exper+tenure`
- For the nonparametric model this just lists the predictors
- For the parametric model it imposes linear structure

Multivariate Nonparametric Regression

```
1 library(np)
2 data(wage1)
3 ghat <- npreg(lwage ~ female + married + educ + exper + tenure, data=wage1, regtype="ll")
4 summary(ghat)
5 ##
6 ## Regression Data: 526 training points, in 5 variable(s)
7 ##               female married   educ   exper   tenure
8 ## Bandwidth(s): 1.058363e-07 0.2881137 5.148991 7.185339 28.00628
9 ##
10 ## Kernel Regression Estimator: Local-Linear
11 ## Bandwidth Type: Fixed
12 ## Residual standard error: 0.36823
13 ## R-squared: 0.520417
14 ##
15 ## Continuous Kernel Type: Second-Order Gaussian
16 ## No. Continuous Explanatory Vars.: 3
17 ##
18 ## Unordered Categorical Kernel Type: Aitchison and Aitken
19 ## No. Unordered Categorical Explanatory Vars.: 2
```

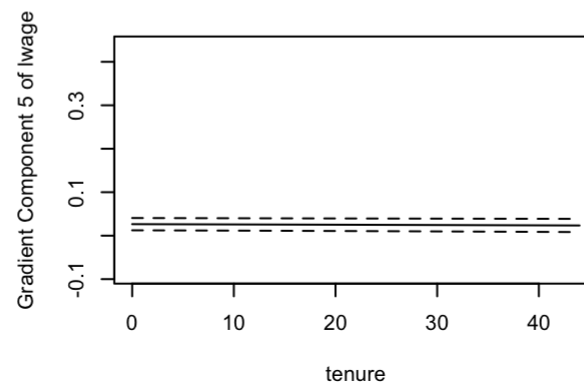
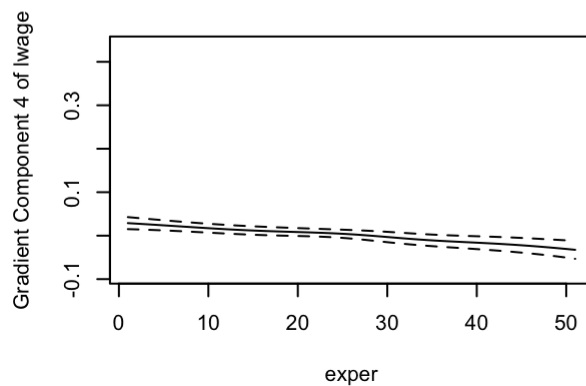
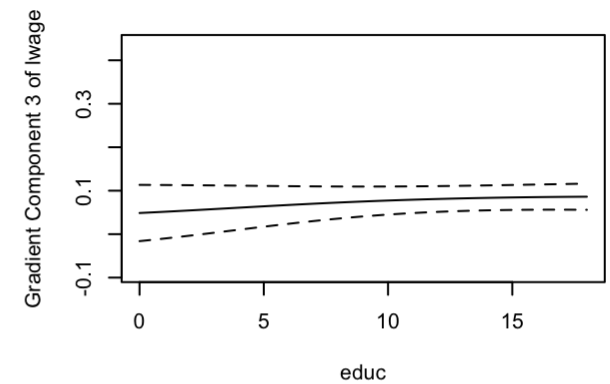
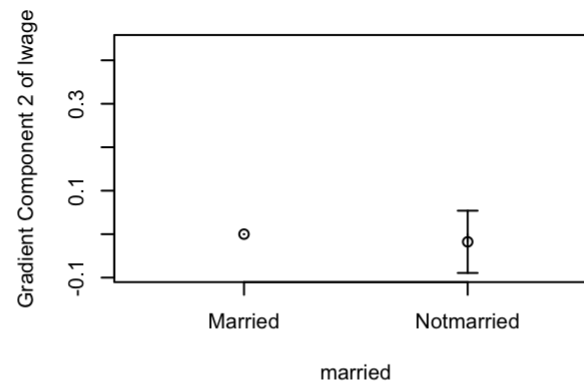
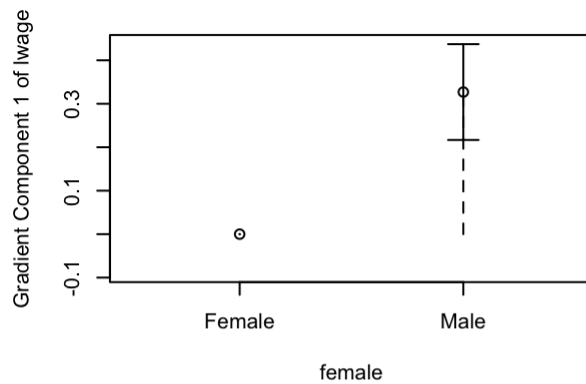
Partial Regression Plots

```
1 ## We run out of graph axis dimensions with > 2 predictors, so it is common to
2 ## construct partial plots that plot the fitted model versus each predictor
3 ## separately holding the off-axis predictors at, say, their median value (you
4 ## can change this - see ?npplot and the argument xq)
5 par(mfrow=c(2,3))
6 plot(ghat, plot.errors.method="bootstrap")
```



Marginal Effects (Gradient) Plots

```
1 par(mfrow=c(2,3))  
2 plot(ghat,gradients=TRUE,plot.errors.method="bootstrap")
```



Testing $H_0: \beta_j = 0$ (OLS Significance)

A simple t -test is used to test $H_0: \beta_j = 0$

```
1 ghat.ols <- lm(lwage ~ female + married + educ + exper + tenure, data=wage1)
2 summary(ghat.ols)
3 ##
4 ## Call:
5 ## lm(formula = lwage ~ female + married + educ + exper + tenure,
6 ##     data = wage1)
7 ##
8 ## Residuals:
9 ##      Min       1Q   Median       3Q      Max
10 ## -1.87254 -0.27256 -0.03779  0.25349  1.23666
11 ##
12 ## Coefficients:
13 ##              Estimate Std. Error t value Pr(>|t|)
14 ## (Intercept)    0.330268   0.106391   3.104  0.00201 **
15 ## femaleMale      0.285530   0.037264   7.662 9.00e-14 ***
16 ## marriedNotmarried -0.125739  0.039986  -3.145  0.00176 **
17 ## educ            0.083905   0.006973  12.033 < 2e-16 ***
18 ## exper           0.003134   0.001682   1.863  0.06300 .
19 ## tenure          0.016867   0.002955   5.707 1.93e-08 ***
20 ## ---
21 ## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
22 ##
23 ## Residual standard error: 0.4125 on 520 degrees of freedom
24 ## Multiple R-squared:  0.4036, Adjusted R-squared:  0.3979
25 ## F-statistic: 70.38 on 5 and 520 DF, p-value: < 2.2e-16
```

Testing $H_0: \beta_j(x) = 0$ (Significance)

- The null and alternative hypotheses are

$$H_0: \beta_j(x) = 0 \text{ for all } x \text{ (a.e.)}$$

$$H_A: \beta_j(x) \neq 0 \text{ for some } x \text{ on a set with } + \text{ measure}$$

- A feasible test statistic $\hat{\lambda} \geq 0$ is given by

$$\hat{\lambda} = \begin{cases} n^{-1} \sum_{i=1}^n \hat{\beta}_j(X_i)^2 & \text{if } x_j \in \mathbb{R} \\ n^{-1} \sum_{i=1}^n \sum_{l=1}^{c-1} \hat{\beta}_j(X_i)^2 & \text{if } x_j \in \mathcal{D} \end{cases}$$

- A bootstrap procedure provides the null distribution

Testing $H_0: \beta_j(x) = 0$ (Significance)

A bootstrap λ -test is used to test $H_0: \beta_j(x) = 0 \forall x$ (a.e.)

```
1 npsigtest(ghat)
2 ##
3 ## Kernel Regression Significance Test
4 ## Type I Test with IID Bootstrap (399 replications, Pivot = TRUE, joint = FALSE)
5 ## Explanatory variables tested for significance:
6 ## female (1), married (2), educ (3), exper (4), tenure (5)
7 ##
8 ##               female    married    educ    exper    tenure
9 ## Bandwidth(s): 1.058363e-07 0.2881137 5.148991 7.185339 28.00628
10 ##
11 ## Individual Significance Tests
12 ## P Value:
13 ## female < 2.22e-16 ***
14 ## married 0.0050125 **
15 ## educ < 2.22e-16 ***
16 ## exper < 2.22e-16 ***
17 ## tenure < 2.22e-16 ***
18 ## ---
19 ## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Counterfactuals

- The R function `fitted(...)` extracts fitted values for each sample observation
- Suppose we want fitted values for *specific value(s)* of the predictor(s)
- We use the R function `predict(..., newdata=...)` where `newdata` points to a data frame containing *named and cast X* values for which we want predictions
- Let's generate a data frame called `df` containing 1 row to generate the predicted log-wage for single males having median education, job tenure and job experience

Counterfactuals

```
1 attach(wage1)
2 df <- data.frame(female = factor("Male", levels=levels(female)),
3                 married = factor("Notmarried", levels=levels(married)),
4                 educ = median(educ),
5                 tenure = median(tenure),
6                 exper = median(exper))
7 head(df)
8 ##      female      married educ tenure exper
9 ## 1   Male Notmarried   12      2   13.5
10 predict(ghat, newdata=df)
11 ## [1] 1.699066
12 ## Or you could use ghat <- npreg(...,newdata=df) and fitted(ghat)
13 ## ghat <- npreg(lwage ~ female + married + educ + exper + tenure,
14 ##              data=wage1,
15 ##              regtype="ll",
16 ##              newdata=df)
17 ## fitted(ghat)
```

If `df` had multiple rows we would get a prediction for each row (i.e., a vector of predictions)

Quarto: Technical, Reproducible Documents

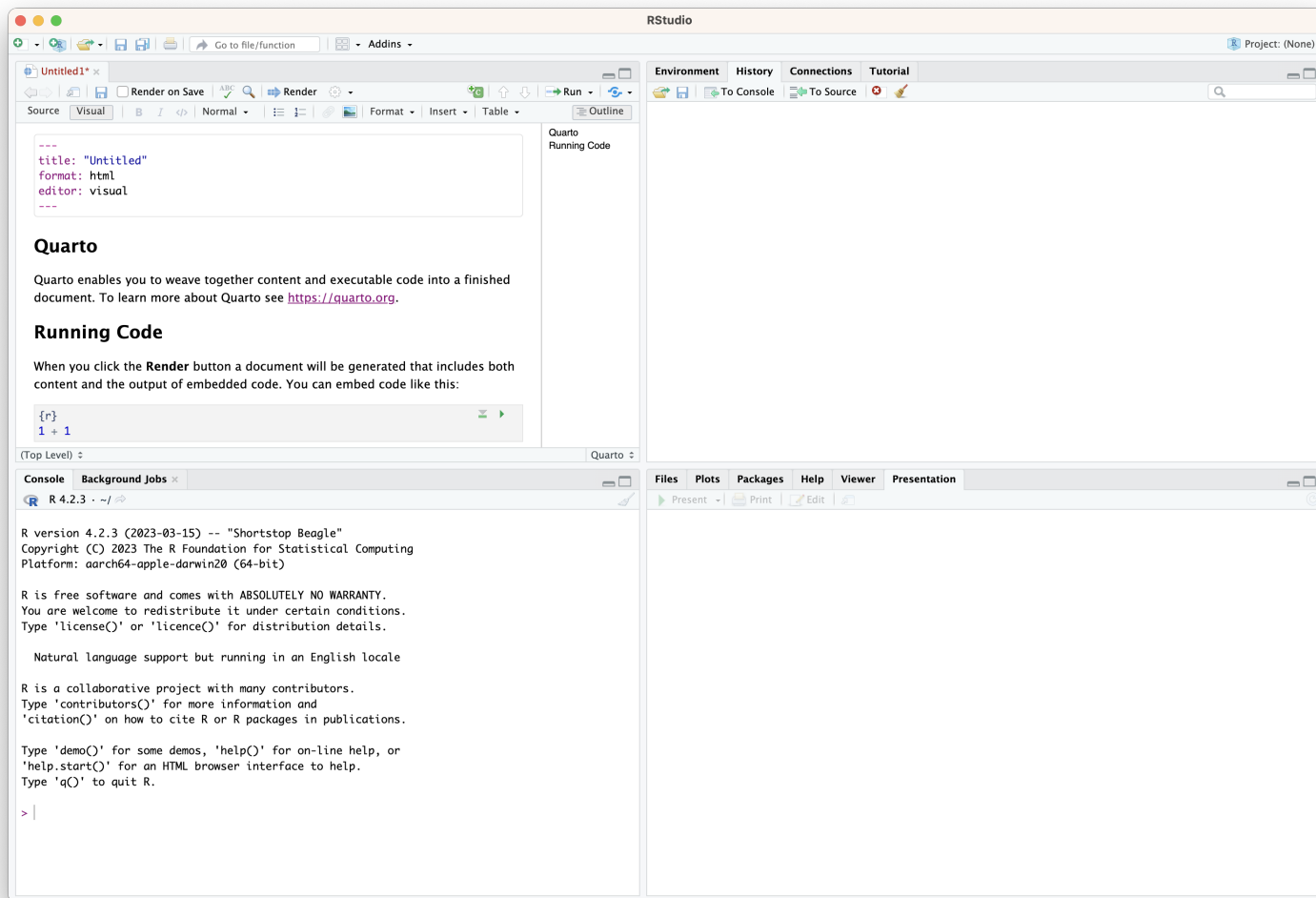
A Sub-Optimal Workflow

- The old way of doing research is to run your analyses using R, then to manually copy figures and tables into a word processor (e.g., MS Word) or typesetter (e.g., T_EX)
- So you end up maintaining separate code, figures, and narrative files and being their go-between coordinator
- And you have pre-committed to an output format!
- Then you find a code or data error and need to redo figures, tables, etc., or need to change output formats
- This is a brutally inefficient and error prone process because it requires a go-between (i.e., you!)

A Streamlined Workflow

- Quarto places your code and narrative into the same file
- Relieved of coordination duties, new possibilities emerge
 - you now *automatically* refer to analytical results
 - you now *conditionally* describe analytical results
- If code or data is updated, figures and tables are automatically updated in your output document
- Perhaps best of all, your Quarto file can be *rendered* into any imaginable output format seamlessly via [Pandoc](#)
- ([Quarto](#), [GitHub](#) and [renv](#) are all supported in [RStudio](#))

RStudio Quarto Editor Interface



Summary

- We have barely scratched the surface and have skipped technical details
- However, you now have some perspective and have access to modern tools
- You can consult Racine ([2019](#)) if you wish to go deeper (detailed proofs, R code, examples etc.)
- Many of the links in these slides may also be useful
- You may contact me at racinej@mcmaster.ca
- It has been my pleasure to guide you! ¡Adiós!

References

- Hayfield, Tristen, and Jeffrey S. Racine. 2008. “Nonparametric Econometrics: The np Package.” *Journal of Statistical Software* 27 (5): 1–32. <https://doi.org/10.18637/jss.v027.i05>.
- Racine, Jeffrey S. 2012. “RStudio: A Platform-Independent IDE for R and Sweave.” *Journal of Applied Econometrics* 27 (1): 167–72. <http://www.jstor.org/stable/41337225>.
- . 2019. *An Introduction to the Advanced Theory and Practice of Nonparametric Econometrics: A Replicable Approach Using R*. Cambridge University Press. <https://doi.org/10.1017/9781108649841>.

